2026 IEEE 26th International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW) | 979-8-3315-7047-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/CCGRIDW69005.2026.00025

Breaking All-Reduce Bottlenecks with Asynchronous Influence Maximization at Scale

Shubhendra Pal Singhal, Ajay Sharma Mandadi, Akihiro Hayashi, Thomas M. Conte, Vivek Sarkar
{ssinghal74, amandadi3, ahayashi, conte, vsarkar}@gatech.edu
Georgia Institute of Technology, Atlanta, USA

***Abstract*—In the era of exa-scale computing, applications involving irregularity and sparsity face non-trivial challenges in achieving scalability. The traditional form of development with the BSP programming model scales poorly due to global synchronization costs and busy-wait times. In this paper, we take a close look at one such big-data graph workload: influence maximization, whose state-of-the-art implementation is a BSP-style Ripples. The problem is to find a set of vertices that have the maximum influence in the graph, and to accomplish this, the algorithm *samples* BFS trees probabilistically in a data-parallel approach, and *selects* the top frequently occurring vertices as influencers using an all-reduce of size equaling the total number of vertices in the graph. The dominant and un-scalable component of the total runtime is this all-reduce in the network subsystem. Therefore, we introduce a novel distributed algorithm Mosaic (Martingale-based Optimized Selection for Asynchronous Independent Cascade) that addresses this scalability problem from two fronts: algorithm design, where we reduce the size of all-reduce to a single integer with no extra memory, and the adoption of a modern fine-grained asynchronous BSP programming model, which enables fine-grained asynchrony and overlap between computation and communication. Mosaic delivers $10\times$ better performance than Ripples on average, and offers scalability, tested up to $131,072$ cores across two different production-grade clusters equipped with InfiniBand and Slingshot-11 interconnects.**

***Index Terms*—FA-BSP, HClib, IMM, Ripples, Mosaic**

I. Introduction

This paper [1] develops a novel distributed algorithm for the problem of influence maximization (IM). Given an input graph, the goal is to identify a set of nodes which have the maximum influence spread over all the remaining nodes of the graph. In other words, the study of consequential virality is known as influence maximization. This problem is prevalent across many critical real-world domains, including public health, economics, supply-chain, viral marketing, and political science [1]–[5]. An example of its primal importance lies in the recent years during COVID-19 pandemic [6], where the governments devised various IM techniques to detect and prevent the spread of the virus.

The influence maximization is a well-known NP-hard problem [7], comprising of two fundamental components: complexity (lower is better) and accuracy (higher is better). To solve this problem, theorists developed a greedy algorithm in 2016 called IMM (Influence Maximization by Martingale) [8], which offered an approximate scalable solution with *theoretical* guarantees of success. Later, in 2019, this algorithm was implemented for distributed CPU clusters in the state-of-the-art Ripples [9] showcasing nearly two orders of magnitude speedup while preserving the accuracy.

The performance gains were impressive, yet they fall short of exploiting parallelism due to its **un-scalable design of**

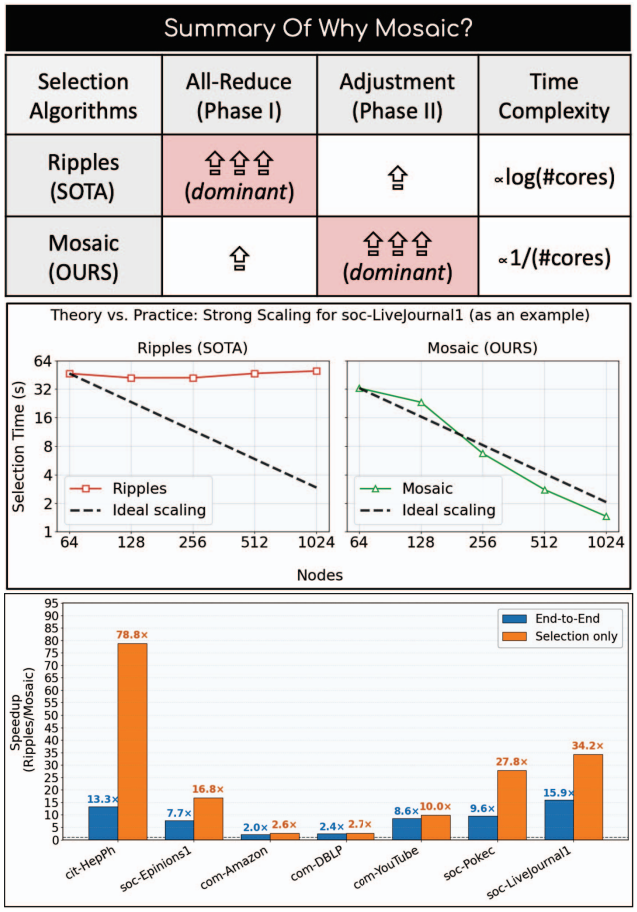


Fig. 1: Summary of the paper. IMM algorithm has two procedures, sampling and selection, out of which selection is dominant on scale, being network bound. It involves two phases: large-sized all-reduce to find the influencer node and subsequent deletion of samples containing this node.

[1]This research is based upon work supported by the Office of the Director of National Intelligence (ODNI), Intelligence Advanced Research Projects Activity (IARPA), through the Advanced Graphical Intelligence Logical Computing Environment (AGILE) research program, under Army Research Office (ARO) contract number W911NF22C0083. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the ODNI, IARPA, or the U.S. Government.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:04:56 UTC from IEEE Xplore. Restrictions apply.

communication sub-system (ref. to ideal scaling and ripples scaling in Fig. 1). The software employs a classical BSP-style (bulk-synchronous parallel) model, wherein each super-step involves an expensive collective (all-reduce) operation, introducing global synchronization costs. In simple terms, the cost of adding more cores to the problem, results in more time being spent in communication, thereby affecting the scalability. This problem becomes **inevitable for high core counts**, since the communication sub-system becomes the dominant part of the overall runtime.

Contributions. In this paper, we attempt to resolve the problem of scalability and performance.

- ξ_1 We *design a new algorithm* Mosaic which reduces the cost of the underlying collective by a factor of number of vertices in the input graph.
- ξ_2 The implementation of Mosaic adopts FA-BSP model (fine-grained asynchronous bulk-synchronous parallel) [10] *which offers scalability* by exploiting the overlap between communication and computation through point-to-point asynchronous, non-blocking messaging.
- ξ_3 We conduct extensive experimentation on *two production-grade clusters for upto 131,072 cores*. Our results reveal that Mosaic performs $\sim 10\times$ better than Ripples (the current state-of-the-art), thereby offering scalability while preserving the accuracy (ref. Fig. 1).

II. Application Scenario

IMM Algorithm. The problem is to find a set of influencers (whose size is denoted by k), from a weighted input graph $G = (V, E, W)$. IMM proposes a greedy strategy to solve this with approximation ratio $(1 - \frac{1}{e} - \epsilon)$ with probability $\geq 1 - \frac{1}{|V|}$ [8], by employing two sub-procedures – sampling and selection in an iterative process. The core concept is to pick a number ρ generated from a function of G, k and ϵ, and sample ρ probabilistic BFS (breadth-first search) trees with different start vertices. We call this procedure sampling. The probabilistic component is a result of classical diffusion model: independent cascade (IC). Under this model, the problem is NP-hard and the decision on bounds of expected influence spread is NP-complete. We refer to the readers to [7] for more details.

Algorithm 1: IMM algorithm

```
1  Input: G, k, ϵ
2  Output: I
3  I ← {}
4  for i ← 1 to log|V| do
5  |  ρ ←f(i, k, ϵ, G) [8]
6  |  S{1..ρ} ← S ∪ Sampling(ρ, G, k)
7  |  I{1..k} ← Selection(S, k)
8  |  coverage ← Σ_{s∈S∩∃v∈{I,s}} 1
9  |  if coverage/|S| ≥ 2^{-i} then
10 |  |  break
11 |  end
12 end
```

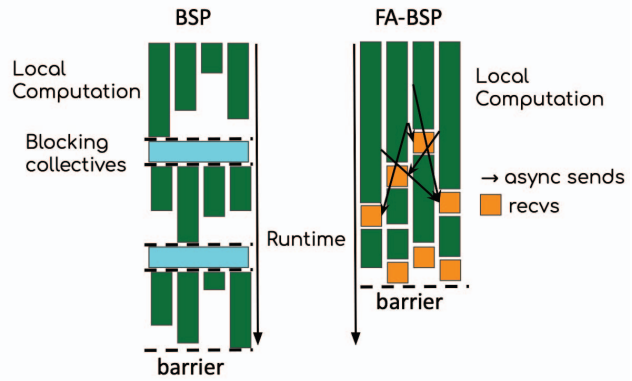


Fig. 2: BSP vs. FA-BSP programming model showcased for 4 compute units.

Next, construct a histogram of occurrence of vertices across all these ρ samples, and pick the vertex v_{infl} with the highest frequency. Then, delete all the samples which contained v_{infl} and repeat the process until k steps. Now, we have a collection of k vertices. This procedure is known as selection. The algorithm then checks the stopping condition which is a function of how many samples were covered by any of the k vertices collected. If the condition satisfies, the algorithm halts, else we repeat the same process of sampling and selection up to $\log |V|$ steps. Refer to Algorithm 1.

How does Ripples parallelize IMM? Ripples replicates the graph onto every compute unit and follows the same algorithmic procedures. In sampling, ripples follows data parallelism, where every core gets an equal number of samples to collect, denoted by ρ/P, where P equals the total core count. Next, selection involves two procedures in k steps, construction of histogram and deletion. Every core first constructs a local histogram from the samples obtained. Then, all cores perform a $|V|$ sized all-reduce operation to obtain a global histogram, containing the frequency of vertices across all samples. Finally, every core locally scans the histogram to obtain the influencer vertex, and performs the deletion of its samples containing this vertex. Note that the deletion process is purely local.

III. Problem Identification and Motivation

The main **challenge associated with ripples is the expensive all-reduce operation**. On smaller set of cores, the sampling procedure constitutes the major part of runtime. On scaling, this procedure scales perfectly. On the other hand, the selection procedure starts off with a lower runtime (relative to sampling). Since all-reduce is the most dominant part of this procedure, *the runtime does not scale at all*, instead rises with a factor of $\log P$ as core count P increases. Therefore, on high scales, the selection procedure becomes the dominant bottleneck. The challenge in achieving perfect scaling is to alleviate the cost of all-reduce operation, and its associated global synchronization costs.

In the next section, we will explain our approach in detail. In synopsis, we exploit an un-used data structure in ripples

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:04:56 UTC from IEEE Xplore. Restrictions apply.

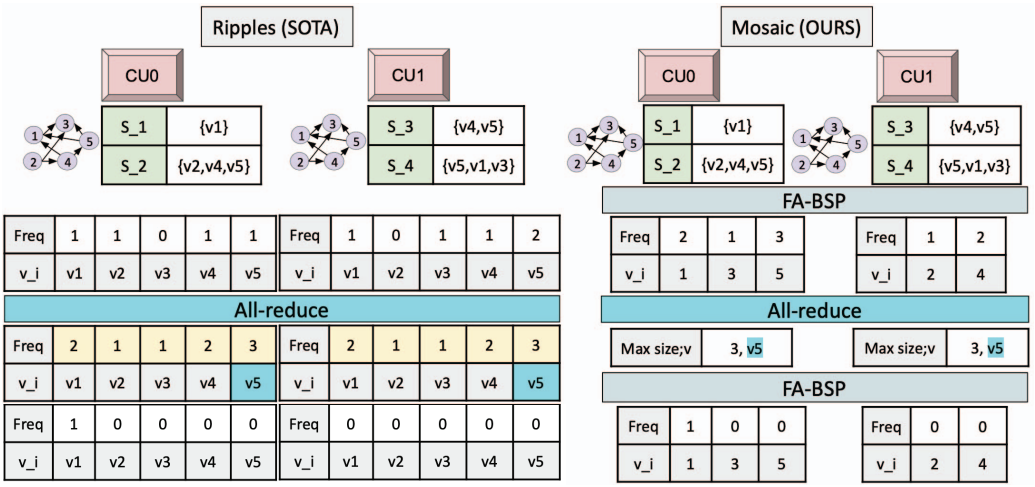


Fig. 3: Ripples (SOTA) vs. Mosaic (OURS). Given a graph G, we show the difference in selection for k=1. The sampling procedure in both algorithms yields the same samples.

to cut the cost of all-reduce by $|V|$, such that only a single-integer all-reduce is required. However, this causes the deletion procedure to not be local anymore. This brings us to the second challenge of implementing the algorithm's communication and computation efficiently. Traditional frameworks with two-sided or one-sided RDMA network calls **suffer from scalability due to the high synchronization costs or high polling time** [10], [11]. Moreover, the **blocking** nature of message passing approaches exacerbates these issues by preventing overlap between communication and computation. Therefore, for exploiting maximum performance, we must *solve for efficient resource utilization* by (1) ensuring near-perfect overlap between computation and communication, (2) using non-blocking asynchronous network calls and (3) exploiting intra-node optimizations, offered by the FA-BSP programming model. We layout the difference of execution in traditional models vs. FA-BSP employed in our implementation in Fig. 2. One must note that usage of single integer all-reduce shifts the bottleneck to the deletion procedure, making it a crucial element of the puzzle to solve.

IV. Our approach: Mosaic

Our algorithm Mosaic (**M**artingale-based **O**ptimized **S**election for **A**synchronous **I**ndependent **C**ascade) follows the same iterative procedure of sampling and selection as IMM. In the next paragraphs, we highlight the differences and similarities between Mosaic and Ripples. Refer to Fig. 3 for an illustrative example. Here, CU denotes compute unit (CU): in Ripples, a CU is an MPI rank, while in Mosaic, a CU is an OpenSHMEM PE (processing element).

Both algorithms load the graph onto every node and follow the same data-parallel sampling procedure, with one key difference in how histograms are maintained and reduced. In Ripples, each CU maintains a replicated $|V|$-length histogram, and the global histogram is obtained by element-wise sum across CUs via an all-reduce over $|V|$ integers. Histogram construction is entirely local on each CU.

In contrast, Mosaic partitions the global histogram by vertex ownership, so each CU maintains counts only for the vertices it owns. For example, in Fig. 3, CU0 maintains counts for v_1, v_3, and v_5, while CU1 maintains counts for v_2 and v_4. During the sampling procedure running on CU i, if it visits a vertex u owned by CU j, the increment must be delivered to CU j. For instance in Fig. 3, CU1 must deliver an increment to CU0 when visiting v_1. This involves inter-CU communication, shown as the first layer implemented using FA-BSP model.

An immediate question is how this design alleviates the all-reduce bottleneck. The key observation is that, once the sampling procedure is over, Mosaic has to find the influencer vertex. Every core scans the histogram and determines the local maximum size present. Then, it performs an integer all-reduce operation to know which size is the maximum, and which vertex did it belong to. This is not possible in ripples approach because the histogram storage is not partitioned; rather every core maintains the histogram array for all vertices.

Looking at the next operation in sequence, Mosaic needs to deletes all the samples which contain the influencer vertex, followed by adjusting the histogram. This sub-procedure is entirely local in ripples, however in our approach, the adjustment of histogram involves communication. Suppose the influencer vertex chosen is v_1. Every core scans the local samples to detect the presence of v_1. Once any sample of such kind is found, the core sends a decrement message to the owner of all vertices in that sample and deletes the sample. For clarity we take an example. Suppose we denote the sample found by $s = \{v_1, v_2, v_3 ... v_m\}$. Then the owner of this sample, sends a decrement message to owners of $v_2, v_3 ... v_m$, wherein upon receiving, the histogram is decremented by one.

Since both sampling and selection require fine-grained point-to-point communication, we adopt the FA-BSP model, watermarked by demonstrated scalability in prior work. For brevity purposes, we refer the reader to [10], [11] for detailed working of FA-BSP model. The tradeoff in performance for

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:04:56 UTC from IEEE Xplore. Restrictions apply.

using our approach vs ripples is the communication associated in the adjustment (or deletion) step vs. an expensive all-reduce operation. The communication component in sampling is negligible, since FA-BSP enables near-perfect overlap by aggregating and sending the messages as soon as visited of a vertex is marked true. The receiving of the messages occurs when the send buffers for an intended destination are full.

Time Complexity. For the assessment of runtime costs, we denote P as total core count, μ as the cost of accessing a unit of local memory, τ as the latency factor for remote communication, and λ as the cost of sending a unit of data to a remote processor. Typically, $\tau > \mu > \lambda$. We note the time taken by selection procedure in both algorithms, since the sampling in both is the same.

For ripples, selection takes $\mathbb{O}\left(k(\tau + \lambda|V|)\log P/c + \mu\sum_{i=1}^{\rho/P}|S_i|\log(|S_i|)\right)$, where c is the processors (cores) per node and S_i denotes the ith sample. The former term in the equation is the traditional all-reduce runtime for $|V|$ size. The latter term is contribution of memory, where the set of samples are scanned, followed by a binary search to check the presence of influencer vertex and lastly, decrementing the histogram for each such sample found. Since due to deletion, a sample can only be scanned once for decrementing the histogram, the term is independent of the factor of k.

In Mosaic, selection takes $\mathbb{O}(k(\tau + 8\lambda)\log P + \lambda\sum_{i=1}^{\rho/P}|S_i| + \mu\sum_{i=1}^{\rho/P}|S_i|\log|S_i|)$. The first two terms concern the network costs. The first is the all-reduce cost of a single-integer. The second term is the cost of communicating all the samples across, throughout the deletion process. This term is independent of k, due to the same property that no sample can be sent twice. The third term corresponds to the memory cost, similar to Ripples.

When does Mosaic win? We observe the two equations provided and derive what makes Mosaic profitable. Equating the two, refer to Equation (1) which when satisfied, results in lower runtime for Mosaic relative to Ripples. Note that, the left term stays constant, as P increases, confirming that Mosaic will deliver more performance and scalability than Ripples.

$$\mathbb{O}(\frac{\sum_{i=1}^{\rho}|S_i|}{k|V|}) < P\log(P/c) \tag{1}$$

Memory Complexity. Both Ripples and Mosaic consume exactly same amount of memory per node with samples being the dominant storage structure, i.e. $\mathbb{O}(\sum_{i=1}^{\rho}|S_i|)$. Refer to Table I. The other data structures required for execution are frontier, visited and histogram which are the same orders as the graph size itself.

V. Implementation of Mosaic

In this section, we list all the technical intricacies of our implementation. First, for the graph representation, we use the CSR (compressed sparse row) format with vertex ids stored in 64-bit format and weights stored in double precision (64-bit) format. Every node loads the graph, forming a replication technique, same as ripples, using `mmap` and `madvise` system calls.

Next, is the the sampling procedure. The data structures employed are visited, frontier, histogram and samples. The visited array is a `std::vector<uint64_t>` of size $\lceil|V|/64\rceil$, where one bit of the vector represents one vertex. This way we compress visited to mere 0.6 MB for the largest dataset in consideration. Next, the frontier is `std::queue` data structure, histogram is a vector of size $|V|/P$ (P being the total core count) and the samples are vector of vectors, whose size is ρ. The vectors inside are dynamic and stores the vertex ids belonging to each sample.

Lastly, the selection procedure involves two sub-steps, all-reduce and deletion. The all-reduce is implemented using OpenSHMEM 1.4 semantics. The deletion phase involves deleting the samples and adjusting the histogram. Instead of deleting the samples which would otherwise take $O(\rho)$ time, we instead create a boolean array of size ρ which on true implies that the sample should not be considered any further (implied deleted). This array is initialized false at the start of k iterations. Moreover, in the adjustment phase, we employ FA-BSP model for communication. We noticed during the implementation that a core repeatedly sends decrement messages to remote vertices, i.e. if say t number of samples contain vertex v, the message -1 will be sent t times. This leads to high communication volume, often witnessed to be $10\times$ higher than the total number of vertices in the graph. Therefore, to avoid this, we include an optimization, we call *"user-chunking"*, where for a fixed hyper-parameter set beforehand, Mosaic will accumulate -1 in an array of size $|V|$, and then flush for sending, when any of the slots becomes higher than the given hyper-parameter. This strategy has been used in the past in [12], [13] and has been proven effective.

VI. Experimentation and Results

We conduct experiments on two clusters: PACE at the Georgia Institute of Technology, using up to 3,072 cores, and Perlmutter at the National Energy Research Scientific Computing Center (NERSC), using up to 131,072 cores. The implementation of FA-BSP model and Mosaic is open-source, available on https://hclib-actor.cc.gatech.edu/ and https://github.com/singhalshubh/actor-imm-ic respectively.

Hardware. PACE has dual-socket Intel Xeon Gold 6226 CPUs clocked at 2.7 GHz with processes per node (ppn) as 24 cores/node, DDR4-2933 MHz DRAM, Infiniband 100HDR interconnect (100 Gb/s), and 96 GB of memory per node. Each CPU node of Perlmutter consists of dual-socket AMD EPYC 7763 (Milan) processors with 128 physical cores per node (ppn), 512 GB of memory, and a single network card of 200 Gb/s connected to a HPE Cray Slingshot-11 interconnect.

Software. Mosaic use OpenSHMEM network layer beneath, with single-threaded multi-process implementation. Ripples use MPI network layer with OpenMP threads. On PACE, we use `python/3.10.10`, `gcc/12.3.0`, and `openmpi/4.1.5` modules. On Perlmutter we use `cray-openshmemx/11.5.8` and

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:04:56 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Datasets, ρ = samples, UD is un-directed, D is directed.

Graph (G)	Nodes	Edges	Size(G)	ρ	Total Size(ρ)
cit-HepPh (D)	35K	422K	8.2 MB	127K	4.2 GB
soc-Epinions1 (D)	76K	509K	9.3 MB	198K	24.6 GB
com-Amazon (UD)	335K	926K	20 MB	123K	209 GB
com-DBLP (UD)	317K	1.05M	22 MB	124K	195 GB
com-Youtube (UD)	1.13M	2.99M	63 MB	185K	607 GB
soc-Pokec (D)	1.63M	30.6M	668 MB	141K	1.19 TB
soc-LiveJournal1 (D)	4.85M	69.0M	1.6 GB	171K	3.55 TB

cray-pmi/6.1.10. On both systems, we place our datasets on the Lustre filesystem for efficient I/O reads. We measure the average execution times for three consecutive runs for all experiments. We report the performance variability of our experiments to be within 1%.

Datasets. Influence Maximization problem is studied for social-network graphs in Ripples. We use the same standard to evaluate Mosaic, shown in Table I. We derive datasets from SNAP benchmark [14] and pre-process them using the 'dump-graph' script from the Ripples [15] library, which labels vertex IDs from $[1..N]$ and generates normalized weights to maintain the apple-to-apple comparison for the same input graph.

A. Outline of Experiments

We then conducted a series of **strong scaling** experiments to study the following aspects of the various implementations,

1) End-to-end runtime of Ripples and Mosaic.
2) Performance of Selection procedure.
3) Accuracy measurement in Ripples and Mosaic

We use PACE for smaller datasets and Perlmutter for larger ones, due to the availability of core counts being higher in the latter. Ripples in [9] showcase performance improvement for $k = 100$ and $\epsilon = 0.13$. We maintain the same arguments for showcasing the performance benefits for Mosaic. We note Mosaic follows the same trends of speedup gains for other values of ϵ and k, and those comparisons are simply not shown in the paper for brevity purposes.

B. End-to-end runtime analysis

We run Ripples and Mosaic on PACE by **scaling out** the workflow from [2...128] nodes with ppn=24 and on Perlmutter from [8...1024] nodes with ppn=128.

PACE. Refer to Fig. 4. We note that for cit-HepPh and soc-Epinions1 graphs, Mosaic performs 4–8× faster and for the rest of the two graphs, Mosaic performs 2× faster than Ripples.

Perlmutter. Refer to Fig. 5. We observe a trend where Ripples doesn't scale well, and the trend continues to worsen as the size of input graph increases. Mosaic performs 8× faster for com-Youtube and 16× faster for the rest of the two graphs, relative to Ripples on the highest core count. We attribute the widening gap on scaling, between the two, to the bottlenecks of all-reduce. Since, the datasets in this figure are relatively larger than the ones used above, we notice the worsening effect of less-to-no scaling for Ripples. On the other hand, Mosaic exploits the asynchrony and overlap between computation and

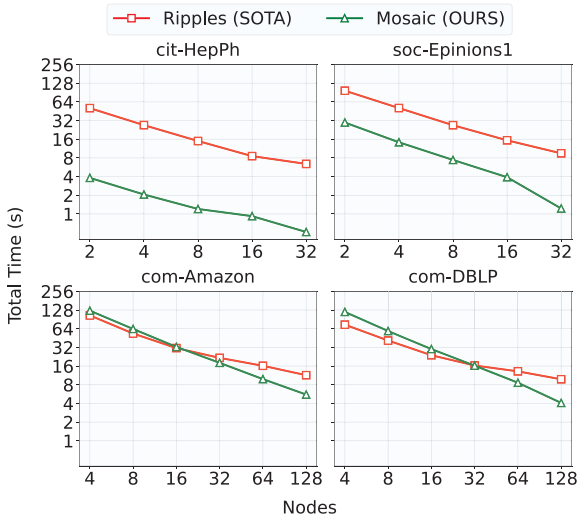


Fig. 4: Strong Scaling for end-to-end comparison for Ripples vs. Mosaic on PACE [ppn=24].

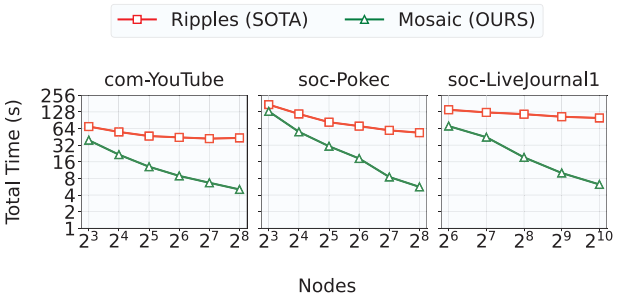


Fig. 5: Strong Scaling for end-to-end comparison for Ripples vs. Mosaic on Perlmutter [ppn=128].

communication offered by the FA-BSP model, showing perfect scaling.

C. Performance of Selection

In this section, we take a look at the selection procedure, which is the only change between Ripples and Mosaic.

PACE. Observe Fig. 6. Mosaic selection procedure runs faster than Ripples by nearly 30× for cit-HepPh, 10× for soc-Epinions1 and 3–4× for the rest of the two graphs. Furthermore, for cit-HepPh and soc-Epinions1, Ripples selection scales linearly since the adjustment phase – not all-reduce phase – dominates the runtime. These two graphs serve as exceptions to showcase that irrespective of which dominates, Mosaic performs better as derived in Equation (1).

Perlmutter. Refer to Fig. 7. Mosaic selection procedure performs 8× faster for com-Youtube, 32× faster for soc-Pokec and 60× faster for soc-LiveJournal1. Since the all-reduce is the dominant portion of the selection procedure in Ripples, we can clearly observe no scaling despite the addition of cores. Moreover, Mosaic scales perfectly, thereby widening the performance gap. To confirm our empirical observation, we

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:04:56 UTC from IEEE Xplore. Restrictions apply.

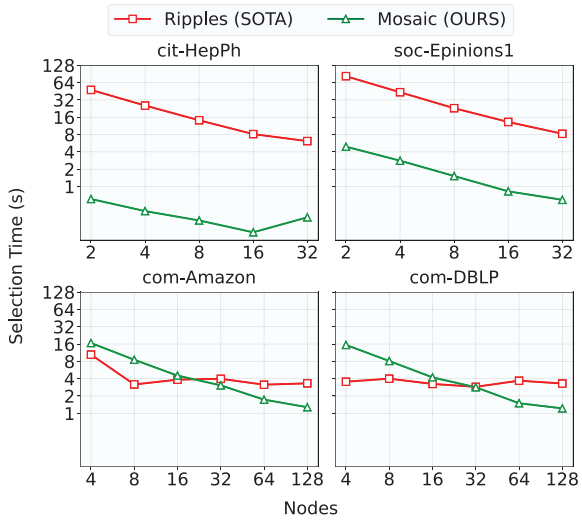


Fig. 6: Strong Scaling for selection procedure for Ripples vs. Mosaic on PACE [ppn=24].

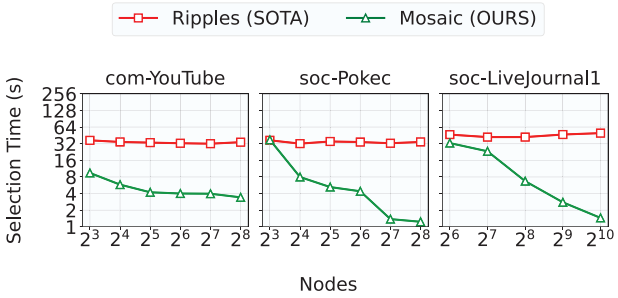


Fig. 7: Strong Scaling for selection procedure for Ripples vs. Mosaic on Perlmutter [ppn=128].

peak at Equation (1), and infer that the theory also predicts the similar runtime trends for the rise in core counts.

Conclusion. We hereby show both in theory and in practice, that Mosaic delivers a significantly higher scalability and performance than the state-of-the-art Ripples, with the geometric mean of end-to-end speedup as $13\times$ for Perlmutter and $3.4\times$ for PACE, across all the datasets.

D. Accuracy of Ripples vs. Mosaic

The accuracy of IMM algorithms is defined as ratio of the samples covered by the set of influencers and the total samples collected. Ripples and Mosaic both follow the same metric for accuracy. Refer to Fig. 8. We observe the same accuracy percentages across both the algorithms with $< 2\%$ deviation. We observe the same for all datasets, not shown in the paper due to brevity purposes.

VII. Related Work

We discuss related work on distributed software for the IMM algorithm and its adoption on different hardware platforms. We first review the developments in the CPU domain that directly correlate with our work, followed by the GPU

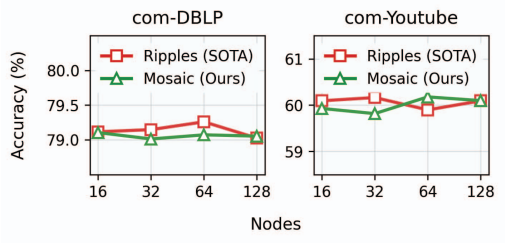


Fig. 8: Comparison of accuracy for Ripples vs. Mosaic

domain. [16] proposed compression of samples to reduce the memory footprint, and [17] introduced new partitioning techniques; however, none of these systems address the scaling bottleneck of the selection procedure. This paper presents the first approach to resolve this bottleneck and enable scalability.

Next, we discuss GPU adoption. Several implementations are available in [18]–[21]: some simply adopt the algorithm across multiple GPUs within a node, while others develop heuristics to simulate the behavior of IMM. Nonetheless, none of these solutions offer scalability beyond a single node. We project the adoption of Mosaic on multi-node GPU setups as the next natural step, since its improved scalability makes such extensions to multi-node environments feasible.

VIII. Impact of Solution

In this paper, we achieve an order of magnitude performance improvement for the problem of influence maximization. The recipe of our scalable solution was to minimize the usage of collectives intertwined with maximizing the usage of fine-grained asynchrony. The former is achieved through a redesign of the parallel algorithm, while the latter is enabled by the FA-BSP programming model.

The broader impact of this work is twofold. **First**, the influence maximization problem is central to today's data-rich world, where meaningful insights are derived from relationships and their propagation dynamics. The precarious nature of applications such as online misinformation, market-for-lemons cascades, and epidemic virality demands immediate intervention due to their potentially catastrophic consequences. Therefore, in domains where relationships govern how disruptions, incentives, or interventions propagate, merely solving the influence maximization problem is insufficient; the solution must execute fast enough to enable timely and accurate control of the outcome. Mosaic delivers both, making it a strong foundation for scalable influence maximization.

Second, the adoption of FA-BSP model in Mosaic demonstrates how to overcome the challenges of exascale computing while maintaining the productivity of the development. The growing use of asynchrony, one-sided RDMA operations, and overlap between computation and communication has once again proven more effective than traditional collective-centric designs. Therefore, this work provides a concrete basis for future research exploring new algorithmic designs and runtime systems that reduce reliance on global collectives while improving scalability at extreme scale for irregular and sparse applications.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:04:56 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. B. Bak-Coleman, I. Kennedy, M. Wack, A. Beers, J. S. Schafer, E. S. Spiro, K. Starbird, and J. D. West, "Combining interventions to reduce the spread of viral misinformation," *Nature Human Behaviour*, vol. 6, no. 10, pp. 1372–1380, 2022.

[2] Q. Ni, Y. Bai, and Z. Tang, "Profit maximization for competitive influence spread in social networks," in *Computing and Combinatorics: 29th International Conference, COCOON 2023, Hawaii, HI, USA, December 15–17, 2023, Proceedings, Part II*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 366–377. [Online]. Available: https://doi.org/10.1007/978-3-031-49193-1_28

[3] M. Azaouzi, W. Mnasri, and L. Ben Romdhane, "New trends in influence maximization models," *Computer Science Review*, vol. 40, p. 100393, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1574013721000332

[4] A. Klages-Mundt and A. Minca, "Optimal intervention in economic networks using influence maximization methods," *European Journal of Operational Research*, vol. 300, no. 3, pp. 1136–1148, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0377221721008894

[5] F. Habibi, R. K. Chakrabortty, and A. Abbasi, "Evaluating supply chain network resilience considering disruption propagation," *Computers & Industrial Engineering*, vol. 183, p. 109531, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0360835223005557

[6] M. Minutoli, R. Neff, N. S. Sattar, H. Lu, J. Feo, H. Mortveit, A. Vullikanti, D. Xie, M. L. Wilson, G. von Laszewski, P. Bhattacharya, S. M. Ferdous, A. Kalyanaraman, M. Becchi, M. Marathe, and M. Halappanavar, "Dimples: Distributed influence maximization for pandemic planning on exascale systems," in *Proceedings of the 39th ACM International Conference on Supercomputing*, ser. ICS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 718–733. [Online]. Available: https://doi.org/10.1145/3721145.3730414

[7] D. Kempe, J. Kleinberg, and E. Tardos, "Maximizing the spread of influence through a social network," in *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 137–146. [Online]. Available: https://doi.org/10.1145/956750.956769

[8] Y. Tang, Y. Shi, and X. Xiao, "Influence maximization in near-linear time: A martingale approach," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 1539–1554. [Online]. Available: https://doi.org/10.1145/2723372.2723734

[9] M. Minutoli, M. Halappanavar, A. Kalyanaraman, A. Sathanur, R. Mcclure, and J. McDermott, "Fast and scalable implementations of influence maximization algorithms," in *2019 IEEE International Conference on Cluster Computing (CLUSTER)*, 2019, pp. 1–12.

[10] S. R. Paul, A. Hayashi, K. Chen, Y. Elmougy, and V. Sarkar, "A fine-grained asynchronous bulk synchronous parallelism model for pgas applications," *Journal of Computational Science*, vol. 69, p. 102014, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877750323000741

[11] F. M. Maley and J. G. DeVinney, "Conveyors for streaming many-to-many communication," in *2019 IEEE/ACM 9th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*, 2019, pp. 1–8.

[12] A. Hayashi, S. P. Singhal, Y. Elmougy, and J. Yang, *Enabling User-Level Asynchronous Tasking in the FA-BSP Model Case Study: Distributed Triangle Counting*. Cham: Springer Nature Switzerland, 2025, pp. 70–77. [Online]. Available: https://doi.org/10.1007/978-3-031-97492-2_8

[13] S. P. Singhal, S. Hati, J. Young, V. Sarkar, A. Hayashi, and R. Vuduc, "Asynchronous distributed-memory parallel algorithms for influence maximization," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–19.

[14] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," http://snap.stanford.edu/data, Jun. 2014.

[15] Marco Minutoli, pnnl/ripples:. Zenodo, Apr. 09, 2021. doi: 10.5281/zenodo.4673587.

[16] X. Chen, M. Minutoli, J. Tian, M. Halappanavar, A. Kalyanaraman, and D. Tao, "Hbmax: Optimizing memory efficiency for parallel influence maximization on multicore architectures," ser. PACT '22. New York, NY, USA: Association for Computing Machinery, 2023, p. 412–425. [Online]. Available: https://doi.org/10.1145/3559009.3569647

[17] R. Barik, M. Minutoli, M. Halappanavar, and A. Kalyanaraman, "Impart: A partitioning-based parallel approach to accelerate influence maximization," in *2022 IEEE 29th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2022, pp. 125–134.

[18] S. Shahrouz, S. Salehkaleybar, and M. Hashemi, "gim: Gpu accelerated ris-based influence maximization algorithm," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 10, pp. 2386–2399, 2021.

[19] G. Göktürk and K. Kaya, "Fast and error-adaptive influence maximization based on count-distinct sketches," *Information Sciences*, vol. 655, p. 119875, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025523014603

[20] G. Göktürk and K. Kaya, "Fast and high-quality influence maximization on multiple gpus," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2022, pp. 908–918.

[21] M. Minutoli, M. Drocco, M. Halappanavar, A. Tumeo, and A. Kalyanaraman, "curipples: influence maximization on multi-gpu systems," in *Proceedings of the 34th ACM International Conference on Supercomputing*, ser. ICS '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3392717.3392750

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:04:56 UTC from IEEE Xplore. Restrictions apply.